PYTHON · OBJECT-ORIENTED PROGRAMMING

Understanding Encapsulation: Protecting Data by Design

A practical guide to bundling data with behavior and restricting direct access to it — covering private attributes, name mangling, validation logic, and real-world modeling with bank accounts, ATMs, and employee records.

Data Hiding | Private Attributes | Name Mangling

Getters & Setters | Validation Logic

Bank Account Case Study | ATM Analogy

Employee Management System

1. What Is Encapsulation?

> Encapsulation is one of the four pillars of object-oriented programming, alongside inheritance, polymorphism, and abstraction. It means bundling data (attributes) and the methods that operate on that data together into a single unit — a class — and restricting direct access to that data by exposing it only through controlled methods.

A useful analogy is a medicine capsule: the medicine sits inside, it cannot be touched directly, and it is only ever used as intended, through the capsule itself. The same idea applies in programming — an object's internal data is sealed inside, and the outside world interacts with it only through a defined set of methods.

1.1 Code Without Encapsulation

Consider a simple `BankAccount` class that stores a balance as a plain, publicly accessible attribute.

bank_account_normal.py

```python
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

account = BankAccount(1000)
print(account.balance)

# Anyone can change it
account.balance = -5000
print(account.balance)
```

OUTPUT

```
1000
-5000
```

1.2 The Problem: No Protection

Nothing prevents an outside piece of code from writing directly to `balance`, even with a nonsensical value:

anywhere_in_the_program.py

```python
account.balance = -100000000
```

Depending on the business rules of the application, a bank account should never be allowed to hold a negative balance like this. Because the attribute is fully exposed, there is no protection — any part of the program, anywhere, can silently corrupt the object's state.

> The core risk: when data is fully public, correctness depends on every single line of code in the entire program behaving responsibly. One careless assignment, anywhere, is enough to break the object.

2. Refactoring With Encapsulation

Instead of allowing direct access, the balance is made “private” by prefixing it with a double underscore, and all interaction happens through dedicated methods:

bank_account_encapsulated.py

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient Balance")

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
account.withdraw(300)
print(account.get_balance())
```

OUTPUT

```
1200
```

Notice that the code never writes `account.__balance = ...` directly. Instead, all changes are routed through methods such as `deposit()` and `withdraw()`, which can validate the request before touching the underlying data.

3. What Happens If Someone Tries to Access It Directly?

direct_access_attempt.py

```
account = BankAccount(1000)
print(account.__balance)
```

OUTPUT

```
AttributeError:
'BankAccount' object has no attribute '__balance'
```

Python hides the attribute from direct outside access, raising an `AttributeError` instead of allowing the read.

3.1 How Python Actually Stores It

Python does not truly make an attribute inaccessible; it performs name mangling. Any attribute prefixed with a double underscore is internally renamed.

name_mangling.py

```
class Bank:
    def __init__(self):
        self.__money = 100
```

Internally, this attribute becomes:

internal representation

```
self._Bank__money
```

This can be confirmed directly:

inspect_dict.py

```
b = Bank()
print(b.__dict__)
```

OUTPUT

```
{'_Bank__money': 100}
```

Technically, the mangled name can still be accessed from outside the class:

bypass_attempt.py

```
print(b._Bank__money)
```

OUTPUT

```
100
```

Strongly discouraged: accessing the mangled name directly bypasses the class's intended interface and reintroduces the exact problem encapsulation is meant to prevent.

4. A Second Example: Student Marks

4.1 Normal Code

student_normal.py

```
class Student:
    def __init__(self, marks):
        self.marks = marks

s = Student(80)
s.marks = 200
print(s.marks)
```

OUTPUT

```
200
```

Problem. Marks should logically stay between 0 and 100, but nothing in this class enforces that constraint.

4.2 Encapsulated Version

student_encapsulated.py

```
class Student:
    def __init__(self, marks):
        self.__marks = marks

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks")

    def get_marks(self):
        return self.__marks

s = Student(80)
s.set_marks(95)
print(s.get_marks())

s.set_marks(150)
print(s.get_marks())
```

OUTPUT

```
95
Invalid marks
95
```

The invalid update is rejected, and the object's previous, valid value is preserved. The object now protects its own data.

5. Visual Comparison

5.1 Without Encapsulation

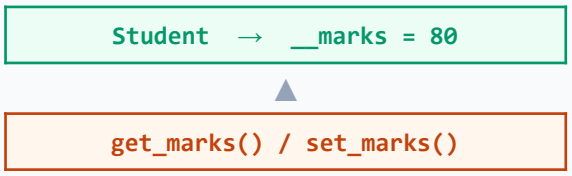
Anyone can directly execute:

anywhere.py

```
student.marks = -50
student.marks = 500
student.marks = "Hello"
```

There is no control over what value is assigned.

5.2 With Encapsulation



All changes must pass through the methods, where the input can be validated.

6. Why Is Encapsulation Useful?

Suppose a class is defined as follows:

car_normal.py

```
class Car:
    def __init__(self):
        self.speed = 0
```

Elsewhere in the program, someone writes:

somewhere_else.py

```
car.speed = -150
```

A negative speed does not make sense. With encapsulation, the same class instead validates the value before accepting it:

car_encapsulated.py

```
class Car:
    def __init__(self):
        self.__speed = 0

    def set_speed(self, speed):
        if speed >= 0:
            self.__speed = speed
        else:
            print("Speed cannot be negative")

    def get_speed(self):
        return self.__speed

car = Car()
car.set_speed(-50)
print(car.get_speed())
```

OUTPUT

```
Speed cannot be negative
0
```

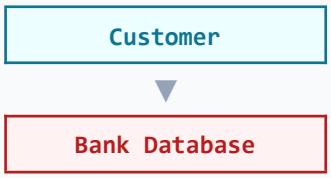
The invalid request is rejected and the object maintains a valid internal state at all times.

6.1 Comparison Table

Feature	Normal Code	Encapsulation
Data access	Direct (obj.balance)	Controlled through methods (get_balance(), deposit(), withdraw())
Validation	Not enforced	Can validate before changing data
Data protection	Weak	Stronger (via name mangling and controlled access)
Risk of invalid data	High	Lower
Ease of maintenance	Harder	Easier, since internal implementation can change without affecting users

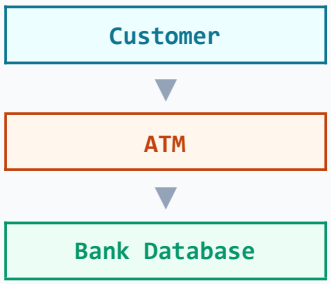
7. A Real-World Analogy: The ATM

7.1 Without Encapsulation



Anyone could directly change the database balance — which is clearly not desirable.

7.2 With Encapsulation



The ATM exposes only controlled operations.

The ATM exposes only controlled operations — withdraw, deposit, and check balance. A customer cannot directly edit the bank's records; the ATM validates each request, for example by checking available funds, before updating the balance. This is exactly how encapsulation works in code: the object's internal data is protected, and interaction happens only through well-defined methods that enforce the rules.

8. A Larger Example: Employee Management System

8.1 Without Encapsulation

employee_normal.py

```
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

emp = Employee("Mehedi", 25, 50000)

# Direct modifications
emp.age = -10
emp.salary = -100000

print(emp.name); print(emp.age); print(emp.salary)
```

OUTPUT

```
Mehedi
-10
-100000
```

Problem. Anyone can freely set `emp.age = -10`, `emp.salary = -100000`, or even `emp.name = ""`. None of these values make sense, and the object has no way to protect itself.

8.2 With Encapsulation

employee_encapsulated.py

```
class Employee:
    def __init__(self, name, age, salary):
        self.__name = name
        self.__age = age
        self.__salary = salary

    # Name methods
    def set_name(self, name):
        if len(name.strip()) > 0:
            self.__name = name
        else:
            print("Name cannot be empty")

    def get_name(self):
        return self.__name

    # Age methods
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Invalid age")

    def get_age(self):
        return self.__age

    # Salary methods
    def set_salary(self, salary):
        if salary >= 0:
            self.__salary = salary
        else:
            print("Salary cannot be negative")

    def get_salary(self):
```

```
        return self.__salary

    def display_info(self):
        print("Name:", self.__name)
        print("Age:", self.__age)
        print("Salary:", self.__salary)

emp = Employee("Mehedi", 25, 50000)
emp.set_age(-10)
emp.set_salary(-100000)
emp.display_info()
```

OUTPUT

```
Invalid age
Salary cannot be negative
Name: Mehedi
Age: 25
Salary: 50000
```

Both invalid updates are rejected, and the employee's original, valid data remains untouched. The invalid data never enters the object.

9. Why This Matters More in Large Programs

Imagine a company codebase with fifty files: employee.py, payroll.py, attendance.py, hr.py, promotion.py, tax.py, and dozens more.

Without encapsulation

anywhere.py

```
emp.salary = -50000
```

This kind of assignment could happen in any one of those fifty files, making a resulting bug very difficult to track down.

With encapsulation

anywhere.py

```
emp.set_salary(-50000)
```

Every salary change must pass through a single method:

employee.py

```
def set_salary(self, salary):
    if salary >= 0:
        self.__salary = salary
```

Result: there is now only one place in the entire codebase where salary can be modified, which makes maintenance, debugging, and future changes far easier.

10. An Even Better Example: Bank Account

10.1 Without Encapsulation

overdraft_normal.py

```
account.balance = 1000
account.balance -= 500
account.balance -= 700
```

RESULT

```
Balance = -200
```

Depending on the bank's policy, overdrafts may not be allowed at all — yet nothing here prevents the balance from going negative.

10.2 With Encapsulation

overdraft_encapsulated.py

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print("Withdraw successful")
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.withdraw(500)
acc.withdraw(700)
print(acc.get_balance())
```

OUTPUT

```
Withdraw successful
Insufficient funds
500
```

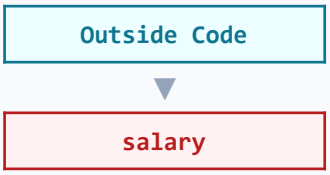
The object enforces its own rules.

11. Key Difference

11.1 Normal Code

normal.py

```
emp.salary = -50000
```

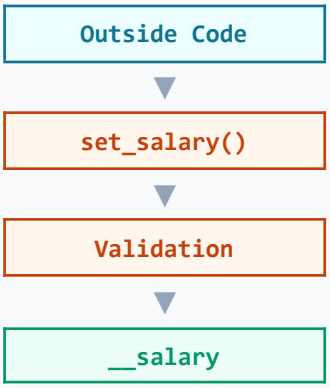


Anyone can change anything.

11.2 Encapsulation

encapsulated.py

```
emp.set_salary(-50000)
```



The method acts like a security guard, checking whether a requested change is allowed before it is applied.

Key takeaway: the main purpose of encapsulation is to hide an object's internal data and force all modifications to pass through controlled methods that enforce validation rules. This keeps an object's state always valid, localizes every possible change to a single, well-defined place in the codebase, and lets the internal implementation evolve freely without breaking the code that depends on it.